UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO

CURSO DE GRADUAÇÃO EM ENGENHARIA ELÉTRICA

ENZO GONÇALVES SEVERO ROCHA
MARIA GABRIELA RODRIGUES GOMES
WESLEY SANTOS NUNES DE OLIVEIRA

PRÁTICA 1: INTERRUPÇÕES

JUAZEIRO - BA

2026

UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO
CURSO DE GRADUAÇÃO EM ENGENHARIA ELÉTRICA

ENZO GONÇALVES SEVERO ROCHA
MARIA GABRIELA RODRIGUES GOMES
WESLEY SANTOS NUNES DE OLIVEIRA

PRÁTICA 1: INTERRUPÇÕES

Relatório apresentado à Universidade Federal do Vale do São Francisco – UNIVASF, Campus Tecnológico, como requisito para obtenção de nota na disciplina de Microcontroladores e Microprocessadores. Orientador: Prof. Dr. Ricardo Prates.

JUAZEIRO- BA

2026

SUMÁRIO

1. INTRODUÇÃO..... 3
2. REFERENCIAL TEÓRICO.....4
2.1 SISTEMA DE INTERRUPÇÕES.....4
2.1.1 Vetor de Interrupção..... 4
2.1.2 Registrador INTCON..... 4
2.2 ROTINA DE SERVIÇO DE INTERRUPÇÃO (ISR).....5
2.3 MÁQUINA DE ESTADOS..... 5
3. METODOLOGIA..... 6
4. RESULTADOS E DISCUSSÕES..... 7
CONSIDERAÇÕES FINAIS..... 9

1. INTRODUÇÃO

O desenvolvimento de sistemas eficientes exige que o microcontrolador seja capaz de reagir a eventos do mundo real em tempo real. Nesse contexto, o microcontrolador PIC16F877A, utilizado em sala de aula, oferece um sistema de interrupções robusto, que permite a interrupção do fluxo principal do programa para o atendimento de eventos prioritários.

Essa prática tem como objetivo a implementação de uma máquina de estados cujo avanço cíclico é controlado, exclusivamente, através da interrupção externa *INT0* no pino *RB0/INT*.

Através do uso do kit de desenvolvimento ACEPIC PRO V2.1 e do gravador Pickit2, o projeto visa consolidar o entendimento prático sobre como interrupções externas podem ser empregadas para gerenciar fluxos lógicos complexos sem a dependência de rotinas de temporização por software.

2. REFERENCIAL TEÓRICO

Durante o experimento, são explorados conceitos fundamentais da programação em Assembly, tais como:

2.1 SISTEMA DE INTERRUPÇÕES

Uma interrupção é um evento que provoca a parada temporária do programa principal para que a CPU atenda a uma tarefa de alta prioridade. No PIC16F877A, existem 15 fontes de interrupções distintas que podem ser habilitadas ou desabilitadas pelo programador.

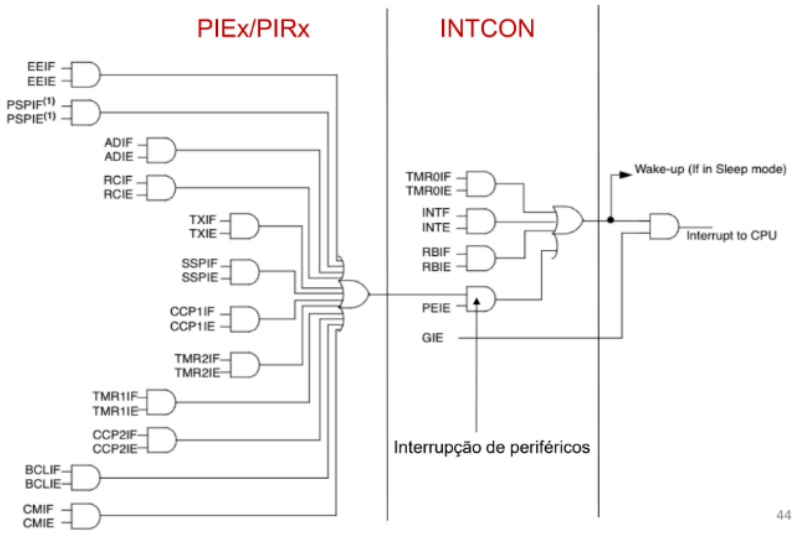
2.1.1 Vetor de Interrupção

Quando uma interrupção ocorre, o microcontrolador salva o endereço atual na pilha e desvia o fluxo para o endereço fixo *0x0004h*.

2.1.2 Registrador *INTCON*

É o principal registrador de controle, contendo bits globais e individuais de habilitação, além das flags de sinalização, conforme mostra a Figura 1:

Figura 1: Diagrama de Blocos de Interrupções



Fonte: Aula 5, Ricardo Prates

- ***GIE* (Global Interrupt Enable):** Chave geral que permite o funcionamento de todas as interrupções habilitadas.
- ***INTE* (INT External Enable):** Habilita especificamente a interrupção no pino *RB0/INT*.
- ***INTF* (INT External Flag):** Bit que é setado quando ocorre o evento no pino, devendo ser limpo dentro da rotina de tratamento para evitar loops infinitos.

2.2 ROTINA DE SERVIÇO DE INTERRUPÇÃO (*ISR*)

A ISR é o bloco destinado a tratar o evento que gerou a interrupção. Um aspecto fundamental nesse processo é o salvamento e restauração de contexto.

- **Preservação de Dados:** Como a interrupção pode ocorrer a qualquer momento, os valores nos registradores *W*, *STATUS* e *PCLATH* podem ser alterados pela *ISR*, o que corrompe o programa principal ao retornar.
- **Instrução *SWAPF*:** É utilizada no salvamento do registrador *STATUS*, garantindo que o estado original do processador seja restaurado fielmente.
- **Instrução *RETFIE*:** Utilizada para finalizar a *ISR*, ela retorna o programa para o endereço salvo na pilha e reabilita automaticamente o bit *GIE.*

2.3 MÁQUINA DE ESTADOS

Uma máquina de estados é um modelo matemático onde o sistema transita entre um número de condições (estados) baseando-se em entradas específicas.

Esta prática possui quatro estados (de 0 a 3), onde a transição ocorre a cada pulso detectado pela interrupção externa. Cada estado define um padrão de acionamento para os LEDs conectados à *PORTD*.

3. METODOLOGIA

Para desenvolvermos este projeto, utilizamos a linguagem Assembly, utilizando o ambiente de compilação MPLAB X IDE. O microcontrolador do nosso experimento foi o PIC16F877A, montado no kit de desenvolvimento ACEPIC PRO V2.1 e gravado com o Pickit2. Na nossa organização de *hardware*, definimos que o pino RB0 (INT) seria a porta de entrada para "escutar" o sinal do botão, enquanto os pinos RD0 a RD3 do PORTD ficariam encarregados de acender os LEDs.

Diferente das abordagens mais simples que prendem o processador em laços infinitos e atrasos, decidimos arquitetar nossa lógica de controle inteiramente em torno do sistema de interrupções. O nosso passo a passo ficou estruturado da seguinte forma:

1. **Primeiro Passo (Setup):** Primeiro, "ligamos a chave geral" de interrupções (bit GIE) e habilitamos a interrupção externa (INTE) no registrador INTCON. O pino RB0 foi configurado como entrada, ficando pronto para reagir assim que o botão fosse pressionado.
2. **O Momento da Interrupção (ISR):** O ponto alto da lógica acontece no vetor de interrupção (endereço `0x0004h`). Quando o acionamento físico ocorre, o microcontrolador pausa o fluxo normal e desvia para essa rotina especial. Lá, nós imediatamente salvamos o contexto (registradores W e STATUS) para não perder nenhuma informação do que a CPU estava fazendo antes.
3. **Avançando a Máquina de Estados:** Dentro da interrupção, o sistema confere se o chamado realmente veio do pino INT (checando a flag `INTF`). Confirmado o aperto, incrementamos a nossa variável de estado (que circula de 0 a 3). Após isso, nós limpamos a flag `INTF` via código para avisar ao sistema que o evento já foi tratado, e devolvemos o controle ao programa principal.
4. **Acendendo as Luzes:** Por fim, o programa principal atua apenas como um observador: ele lê a variável de estado e aplica os comandos de ligar e desligar (`bsf` e `bcf`) nos pinos do PORTD, sempre respeitando a lógica inversa exigida pelas conexões eletrônicas da nossa placa.

4. RESULTADOS E DISCUSSÕES

A implementação da máquina de estados controlada via interrupção externa (INT0) atendeu plenamente à lógica proposta. O sistema digital foi capaz de realizar o mapeamento entre as quatro condições exigidas: o Estado 0 acionou RD0 e RD2; o Estado 1 acionou RD1; o Estado 2 acionou RD2 e RD3; e o Estado 3 acionou RD3.

A grande e mais importante discussão técnica desta implementação, no entanto, gira em torno do clássico efeito de repique mecânico, o *bouncing*. Como o nosso objetivo primário foi criar um código eficiente, liberando a CPU e eliminando qualquer rotina de *delay*, nós retiramos intencionalmente os atrasos que costumavam "mascarar" as trepidações do botão.

Na prática, mostrou que quando pressionamos um botão, seus contatos metálicos internos não se fecham de forma perfeita e limpa em um único instante. Eles colidem e "quicam" microscópica e eletricamente por alguns milissegundos antes de estabilizar.

O resultado direto da ausência do *delay* por software é que, sem esse filtro de tempo, a máquina de estados pode se tornar instável na prática física. Um único toque no botão pode gerar três ou quatro interrupções em frações de segundo, fazendo com que o sistema salte estados rapidamente (por exemplo, pulando do Estado 0 direto para o Estado 2).

Esse comportamento não é um erro lógico, mas sim a revelação de um compromisso fundamental em sistemas embarcados. Sem o atraso do código, ficou claro que um projeto responsivo e limpo exigiria a adição de um filtro (um circuito RC) diretamente no pino do botão para suavizar o sinal elétrico antes de ele chegar ao microcontrolador, ou, alternativamente, o uso mais avançado de um *Timer* rodando em segundo plano.

Apesar deste desafio prático do *bouncing*, a substituição da varredura contínua (*polling*) pelas interrupções cumpriu seu papel arquitetural. Além disso, o emprego de comandos em linguagem baixo nível demonstrou a correta aplicação da

lógica inversa do *hardware* do kit, ativando os LEDs com nível baixo (0) com sucesso.

CONSIDERAÇÕES FINAIS

A prática cumpriu com êxito o objetivo de consolidar o entendimento sobre o sistema de interrupções e o fluxo de instruções no PIC16F877A. A implementação de uma máquina de estados com avanço cíclico controlado unicamente pela interrupção INT0 evidenciou a capacidade do microcontrolador de gerenciar tarefas prioritárias de forma assíncrona.

Conclui-se que o domínio da programação em baixo nível (Assembly) para a configuração de registradores de controle, salvamento de contexto e tratamento de flags é um conhecimento indispensável no desenvolvimento de sistemas embarcados eficientes e de alta responsividade.